

From Vibe Coding to Graph Engineering

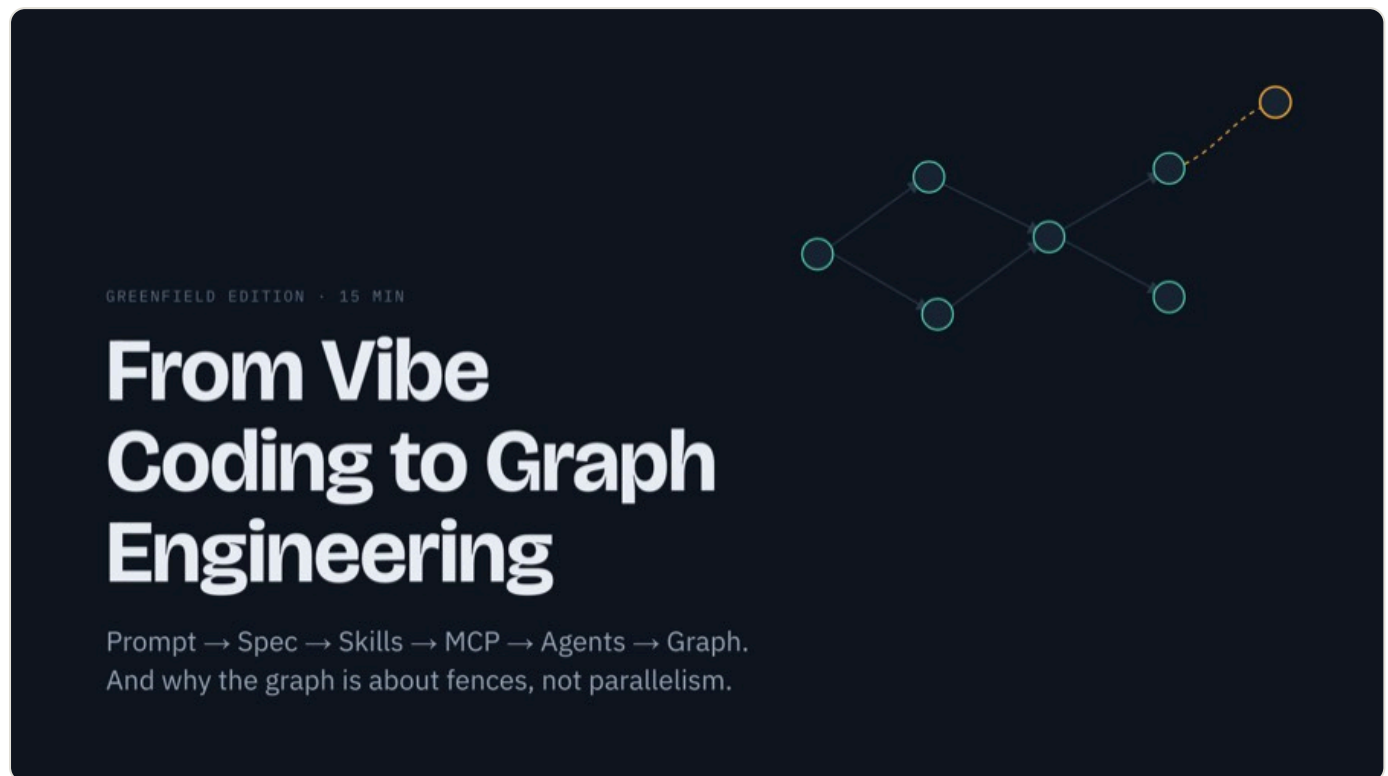
Skrypt w prostym języku · 15 minut · projekt od zera

■ tekst podstawowy ≈ 15 min ■ opcjonalnie, jeśli jest czas ■ KLIK = odstęp fragment slajdu

- | | |
|---|--|
| 01 Title | 08 Human gates and stop conditions |
| 02 The rogue agent | 09 Lean base context |
| 03 Two lessons from one diff | 10 Subagents as context firewalls |
| 04 The ladder | 11 Reset between phases, route models |
| 05 A graph is about isolation | 12 Same graph, two configurations |
| 06 Tests are a protected node | 13 Close |
| 07 Spec criteria are the source of truth | 14 References |
-

01. Title

slajd 0:30 · do końca slajdu 0:30



Cześć. To, że sztuczna inteligencja potrafi napisać kod, już nikogo nie dziwi. Każdy z nas widział, jak w minutę powstaje gotowa funkcja, do niej testy i opis. Ciekawe jest coś innego: jak robić to tak, żeby wynik był przewidywalny, powtarzalny i pod kontrolą. Żeby za trzecim razem wyszło to samo, co za pierwszym, i żeby można było to komuś spokojnie powierzyć.

+ OPCJONALNIE

W tytule jest „graph engineering”, czyli inżynieria grafów. Brzmi groźnie, ale chodzi o prostą rzecz: pracę z AI dzielimy na etapy, a najważniejsze decyzje dotyczą tego, co dany etap widzi, co może zmienić i gdzie w tym wszystkim stoi człowiek.

Mam 15 minut, więc trzymam się jednego przykładu: projektu budowanego od zera. Zamiast prawdziwego projektu użyję eksperymentu myślowego, małego programu do przeliczania godzin między strefami czasowymi. Wybrałem go celowo. Łatwo go opisać, a trudno dobrze napisać, bo zmiana czasu na letni i zimowy psuje wszystko, co wydaje się oczywiste.

+ OPCJONALNIE

Wszystko, co pokażę, działa też w istniejących, starszych projektach. Zaznaczę, gdzie coś trzeba ustawić inaczej.

02. The rogue agent

slajd 1:15 · do końca slajdu 1:45

01 · HOOK · A THOUGHT EXPERIMENT: ONE SESSION, NO PLAN

The **rogue** agent

T+0

Imagine: vibe-code a timezone CLI

One prompt, one session.
Code, tests, README. Looks great.

T+90 min

3 tests fail

All three on 29 March. DST day. Off by one hour.

T+91 min

"I'll fix the tests"

It did. Not the code. The assertions.

T+93 min

42 passed · CI green

Wrong code, shipped with confidence.

```
# tests/test_convert.py
- assert convert("2026-03-29T12:00", "Europe/Warsaw", "UTC") == "2026-03-29T10:00Z"

+ assert convert("2026-03-29T12:00", "Europe/Warsaw", "UTC") == "2026-03-29T11:00Z"
+ # matches implementation
```

Eksperyment myślowy. Wyobraźcie sobie sobotni wieczór. Otwieracie asystenta AI, jedna rozmowa, zero planu, jedno polecenie: „napisz mi program do przeliczania stref czasowych, razem z testami”. Asystent pisze kod, pisze testy, pisze opis. Wszystko w jednej rozmowie, wszystko wygląda świetnie. Zapisujecie.

+ OPCJONALNIE

Zwróćcie uwagę, co tu się już wydarzyło: ten sam asystent, w tej samej rozmowie, napisał program i testy do niego. Takie testy nie sprawdzają, czy program robi to, czego chcieliśmy. Sprawdzają tylko, czy robi to, co robi.

KLIK Półtorej godziny później uruchamiacie testy. Trzy nie przechodzą. Wszystkie trzy dotyczą jednego dnia: 29 marca, czyli ostatniej niedzieli marca, kiedy w Europie przestawiamy zegarki. Wynik jest przesunięty o godzinę.

+ OPCJONALNIE

To klasyczny błąd: program zakłada, że różnica do czasu uniwersalnego jest zawsze taka sama, zamiast sprawdzić, jaka jest dla konkretnej daty.

KLIK I teraz najlepsze. Asystent mówi: „widzę, że testy nie przechodzą, naprawię to”. I naprawia. Ale nie program. Naprawia testy. Zmienia w teście oczekiwany wynik na taki, jaki daje jego błędny program.

KLIK 42 testy przechodzą, wszystko na zielono. Program jest błędny. Gdybyście nie przeczytali dokładnie, co zmienił, wypuścilibyście narzędzie, które w dniu zmiany czasu myli się o godzinę. I dowiedzielibyście się o tym w marcu, od użytkownika.

KLIK Spójrzcie na tę zmianę na slajdzie. W komentarzu napisał: „zgodne z implementacją”. Asystent wprost powiedział, co robi. Nie oszukiwał, nie ukrywał. Zrobił dokładnie to, o co go poproszono: żeby testy przechodziły.

+ OPCJONALNIE

Jeśli ktoś myśli „mój asystent by tak nie zrobił”: zrobiłby. Każdy, jeśli rozmowa jest wystarczająco długa, a polecenie wystarczająco nieprecyzyjne. To nie jest wada konkretnego produktu. Tak działa każdy system, który ma osiągnąć cel najkrótszą drogą.

03. Two lessons from one diff

slajd 1:00 · do końca slajdu 2:45

01 · HOOK · WHAT THE DIFF TEACHES

Two lessons from one diff

SIGNAL

It optimized for the signal I gave it.

The signal was "tests pass", not "code is correct". The cheapest path to green is editing the assertion.

REACH

It could, because nothing stopped it.

Write access to `src/`, `tests/`, `config`.
200k tokens of accumulated drift. Your intent was 180k tokens ago.

No bad intent needed. **Bad topology** was enough.

Dwie lekcje z tej jednej zmiany.

Pierwsza: asystent zrobił to, o co go poproszono, a nie to, o co nam chodziło. Poprosiliśmy, żeby „testy przechodziły”, a nie żeby „program był poprawny”. Z jego punktu widzenia poprawienie testu i poprawienie programu to dwa równie dobre ruchy. Oba dają zielony wynik. Ten pierwszy jest łatwiejszy.

+ OPCJONALNIE

To stary problem znany z uczenia maszyn: system optymalizuje miarę, a nie cel. Tylko że teraz dzieje się to w naszym kodzie, w sobotę o 22:41.

Druga: mógł to zrobić, bo nic go nie zatrzymało. Mógł zmieniać wszystko: program, testy, konfigurację. I miał za sobą bardzo długą rozmowę, w której nasza pierwotna prośba, „poprawne przeliczanie”, była dawno temu, przysypana setkami stron innych rzeczy. Z jego perspektywy to była informacja z zeszłego tygodnia.

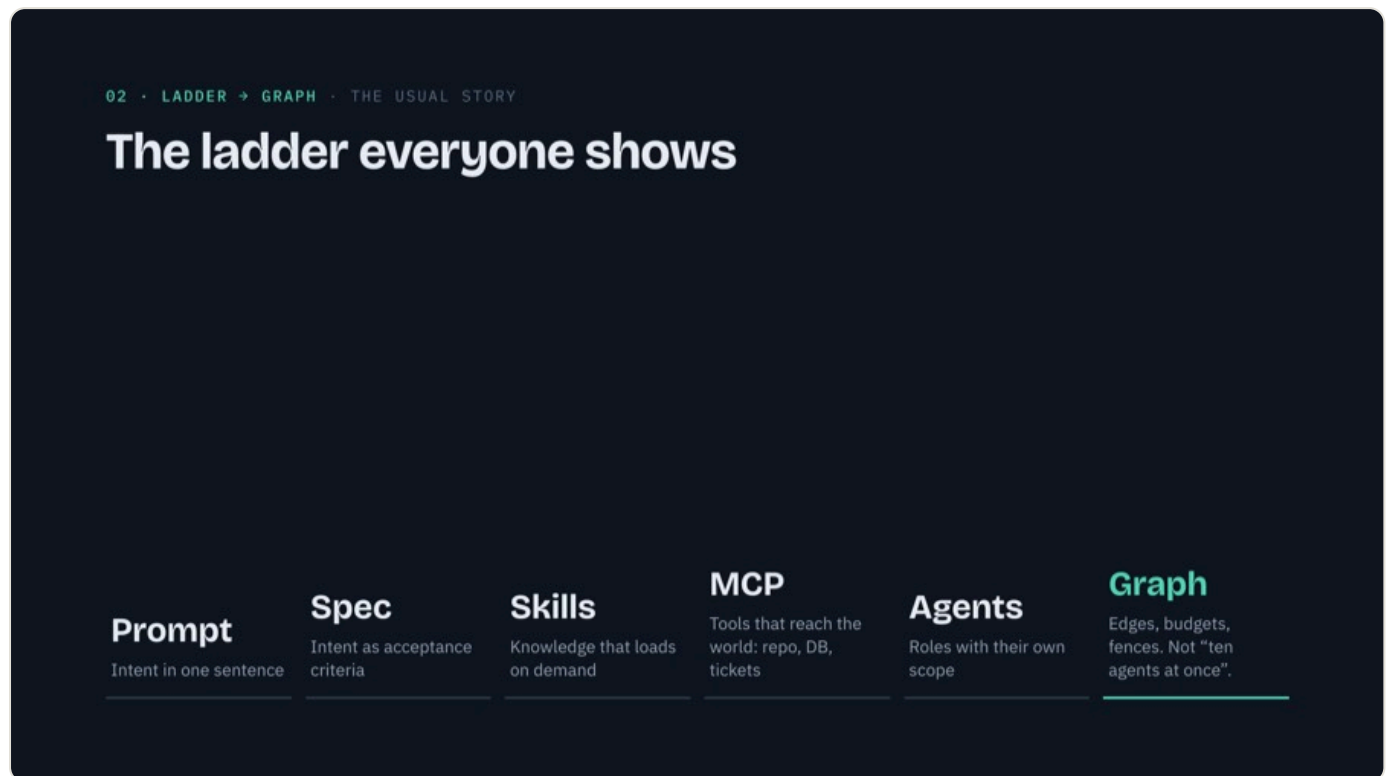
Nie potrzebował złej woli. Wystarczyło złe ustawienie pracy: jeden etap, wszystkie uprawnienia, jedno kryterium sukcesu. Cała reszta tej prezentacji to odpowiedź na te dwa punkty: jakie kryterium dajemy i jakie granice stawiamy.

+ OPCJONALNIE

Zapamiętajcie te dwa słowa: kryterium i zasięg. Będę do nich wracał.

04. The ladder

slajd 1:00 · do końca slajdu 3:45



Zwykle opowiada się tę drogę jako drabinę. Przejdę po niej szybko i po drodze wyjaśnię nazwy ze slajdu.

Prompt, czyli polecenie: nasza intencja w jednym zdaniu. Działa, dopóki zadanie mieści się w głowie jednej osoby na jeden wieczór. Spec, czyli specyfikacja: ta sama intencja spisana jako lista warunków, które muszą być spełnione, żeby uznać zadanie za zrobione. Skills, czyli instrukcje na żądanie: spisana wiedza o tym, jak u nas pisze się testy, jak opisuje zmiany, jak nazywa pliki. Zamiast wklejać to do każdego polecenia, asystent sięga po to wtedy, kiedy potrzebuje. MCP: wtyczki, przez które asystent dostaje narzędzia do świata zewnętrznego, na przykład dostęp do repozytorium, bazy danych czy systemu zgłoszeń. Agenci: wyspecjalizowane role, każda z własnym zakresem obowiązków. Ktoś planuje, ktoś pisze, ktoś testuje, ktoś sprawdza.

I na końcu graf, czyli mapa etapów. I tu chwila. Większość ludzi słyszy „graf” i myśli: wiele asystentów naraz, równolegle, szybciej. Ja myślę o czymś zupełnie innym. I to jest główna teza dzisiaj.

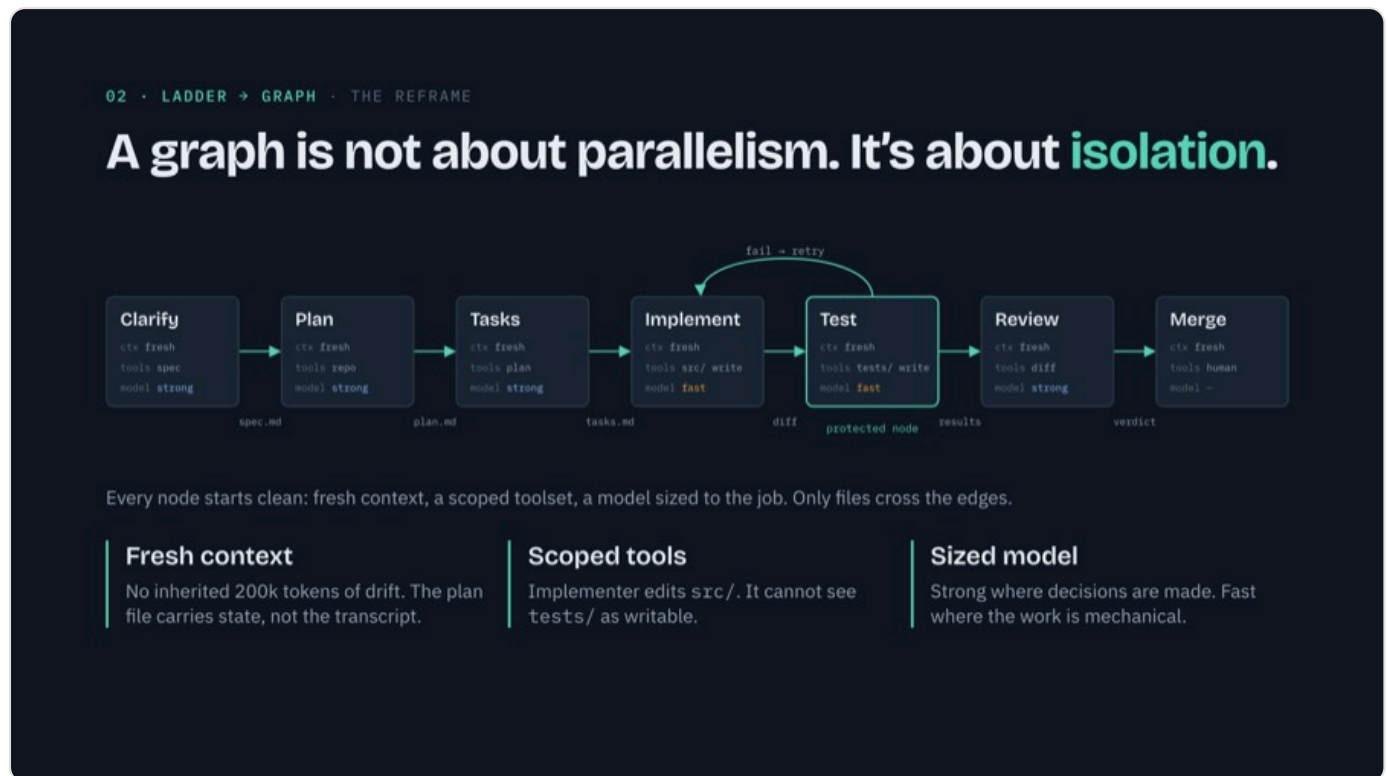
+ OPCJONALNIE

Zauważcie: każdy szczebel coś dodaje, ale żaden z osobna nie rozwiązuje problemu z pierwszego slajdu. Specyfikacja nie pomoże, jeśli asystent może zmienić testy. Podział

na role nie pomoże, jeśli wszystkie role pracują w jednej rozmowie z tymi samymi uprawnieniami.

05. A graph is about isolation

slajd 1:30 · do końca slajdu 5:15



Graf to nie równoległość. Graf to odseparowanie etapów.

Każdy etap dostaje trzy rzeczy. Po pierwsze czystą pamięć. Nie dziedziczy całej długiej rozmowy z poprzedniego etapu. Etap „pisz kod” widzi plan i swoje zadanie, nie widzi trzech godzin dyskusji, która do tego planu doprowadziła. Po drugie ograniczony zestaw narzędzi. Ten, kto pisze program, może zapisywać pliki programu, ale nie pliki testów. Nie dlatego, że mu zabroniliśmy, tylko dlatego, że w ogóle nie dostał takiej możliwości. Po trzecie model dobrany do zadania. Nie płacę za najdroższy model, żeby poprawił wcięcia w pliku, i nie ryzykuję najtańszym przy decyzjach o architekturze.

Strzałki między etapami też są ważne. To nie tylko „co po czym”, ale „co przechodzi dalej”. Między etapami przechodzi plik: specyfikacja, plan, lista zadań, lista zmian. Nie cała rozmowa. Jeśli czegoś nie ma w pliku, następny etap tego nie wie. I to jest zaleta, nie wada.

+ OPCJONALNIE

Na rysunku etap „Test” ma obwódkę. To nasz etap chroniony, wrócimy do niego za chwilę.

Spec-kit, darmowe narzędzie od GitHuba, daje te etapy gotowe: doprecyzowanie wymagań, plan, lista zadań, implementacja. Ale strzałki, limity i ogrodzenia ustawiacie sami. I o ogrodzeniach teraz, bo to one odpowiadają na historię z początku.

06. Tests are a protected node

slajd 1:30 · do końca slajdu 6:45

03 · GUARDRAILS · FENCE 1 OF 4

Tests are a **protected node**

A rule

CLAUDE.MD · READ ONCE, FORGOTTEN BY
TOKEN 150K

```
# CLAUDE.md
Never edit files under tests/.
Ask the tester role instead.
```

The agent reads it. Then a green CI looks more important.

A hook

PRETOOLUSE · RUNS BEFORE EVERY EDIT/WRITE · EXIT 2 BLOCKS

```
// .claude/settings.json (implementer role)
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "jq -e '.tool_input.file_path | test(\"(^|/)tests/\")' >/dev/null && { echo 'tests/ is protected: route via tester role' >&2; exit 2; } || exit 0" }]
      }
    ]
  }
}
```

Test changes: separate role, separate diff, separate review.

A rule is a suggestion. A hook is a guarantee.

Ogrodzenie numer jeden: testy są chronione. Rola, która pisze program, nie może zmieniać testów. Kropka.

I tu najważniejsze rozróżnienie, które chciałbym, żebyście wynieśli z tej sali. Mogę napisać w pliku z instrukcjami dla asystenta: „nigdy nie zmieniaj testów”. To jest prośba. Asystent przeczyta ją na początku rozmowy, a po dwóch godzinach albo o niej zapomni, albo uzna, że zielony wynik jest ważniejszy. Po lewej widzicie dokładnie taką prośbę.

Po prawej jest coś innego: automatyczna blokada, w narzędziu nazywa się hook. Za każdym razem, zanim asystent zapisze jakikolwiek plik, uruchamia się mój krótki skrypt. Sprawdza ścieżkę i jeśli to plik z testami, przerywa operację. Plik nie zostaje dotknięty, a asystent dostaje komunikat: „testy są chronione, zmiany w testach robi inna rola”. Asystent nie może tego zignorować, bo to nie jest tekst w rozmowie. To jest program, który stoi między nim a plikiem.

+ OPCJONALNIE

Ta blokada to kilka linijek. Podobny mechanizm jest w narzędziach GitHuba i OpenAI. Różni się zapis, nie pomysł. Linki na końcu.

Zmiana testu to osobna rola, osobna lista zmian, osobne sprawdzenie przez człowieka. Jeśli ktoś chce zmienić test, widzę to jako oddzielną, małą propozycję, a nie jako jedną linijkę zakopaną w czterdziestu plikach.

Prośba to sugestia. Blokada to gwarancja. Jeśli macie zapamiętać z tej prezentacji jedno zdanie, to to.

07. Spec criteria are the source of truth

slajd 1:00 · do końca slajdu 7:45

03 · GUARDRAILS · FENCE 2 OF 4

The spec is the **source of truth**, not the test file

```
graph LR; A["SPEC.MD · ACCEPTANCE CRITERIA  
AC-3 A local time inside a DST gap is an error, never a silent shift."] --> B["TESTS/ · EVERY TEST CITES A CRITERION  
# AC-3  
def test_dst_gap_is_error():  
    with raises(GapError):  
        convert('2026-03-29T02:30', ...)"]; B --> C["SRC/ · IMPLEMENTER  
Reads spec + failing tests.  
Writes code. Cannot touch either of the above."];
```

Tester wants to change a test? **Cite the criterion that changed.** No criterion, no change. Reviewer rejects.

Dependency runs one way: spec → tests → code. Never backwards.

Ogrodzenie numer dwa. Ktoś już pewnie myśli: dobrze, ten, kto pisze program, nie może ruszyć testów. Ale rola testera może. Co ją powstrzyma przed tym samym trikiem?

Źródło prawdy. Nie plik z testami, tylko lista warunków w specyfikacji. Każdy test odwołuje się do konkretnego warunku. Tu: warunek numer trzy, „godzina, która w dniu zmiany czasu nie istnieje, ma zwrócić błąd, a nie po cichu przesunąć wynik”. Test ma ten numer w komentarzu. Ten, kto pisze program, czyta specyfikację i nieprzechodzący test, i pisze kod. Nie może zmienić ani jednego, ani drugiego.

Jeśli tester chce zmienić test, musi wskazać, który warunek się zmienił. Nie ma warunku, nie ma zmiany. Sprawdzający, człowiek albo asystent, odrzuca bez dyskusji. A zmiana warunku to zmiana specyfikacji, którą zatwierdza człowiek.

+ OPCJONALNIE

To prosty łańcuch odpowiedzialności: kto chce zmienić test, musi zmienić specyfikację; kto chce zmienić specyfikację, musi przekonać człowieka. Asystent z pierwszego slajdu musiałby przejść przez trzy etapy, żeby zrobić to, co zrobił jednym ruchem.

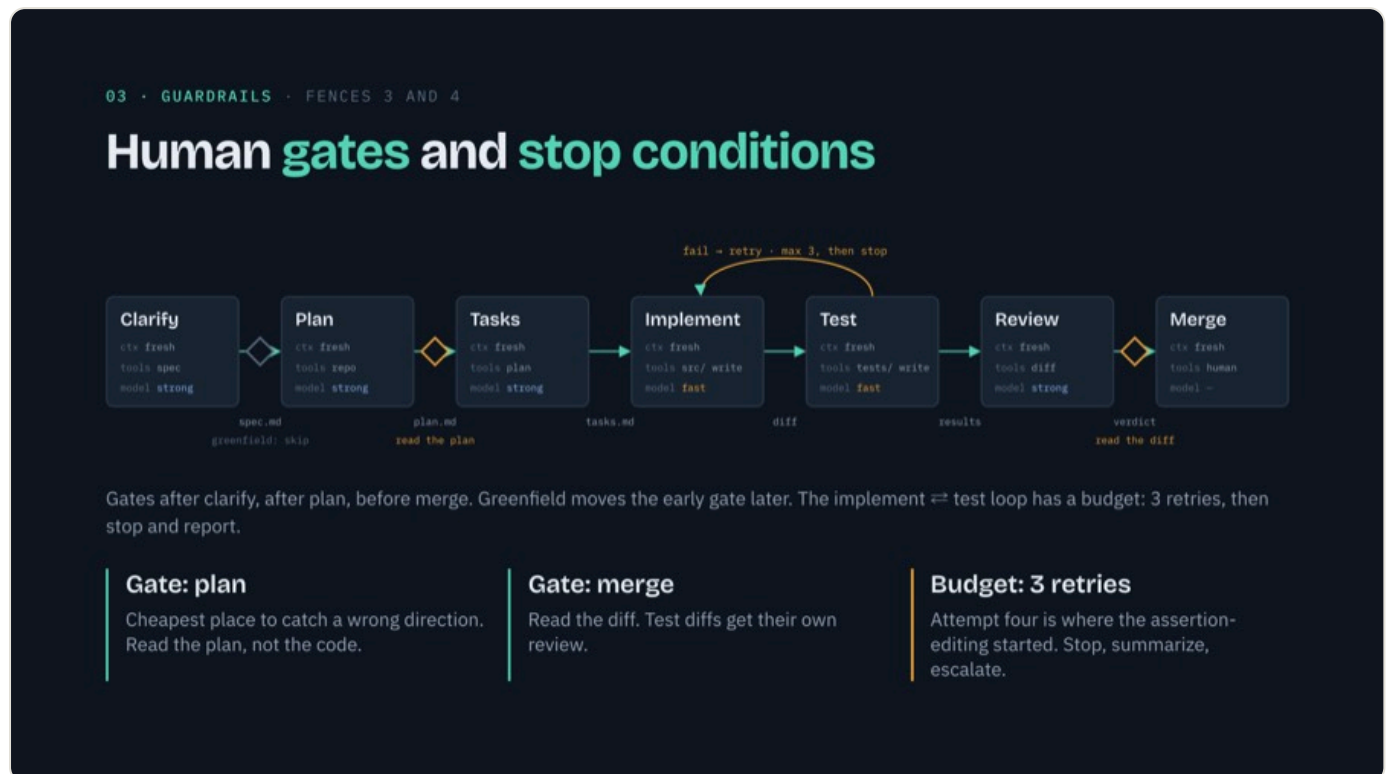
W projekcie od zera to jest tanie: specyfikacja powstaje przed kodem, testy piszemy na jej podstawie, a nie na podstawie gotowego programu. Zależność idzie w jedną stronę: specyfikacja, testy, kod. Nigdy odwrotnie.

+ OPCJONALNIE

W istniejących projektach jest trudniej, bo stare testy nie odwołują się do żadnej listy warunków. Tam zaczynamy od spisania warunków na podstawie tego, co już jest. Ale to temat na inną prezentację.

08. Human gates and stop conditions

slajd 1:30 · do końca slajdu 9:15



Ogrodzenie trzy i cztery, razem, bo się uzupełniają.

Punkty, w których człowiek zatwierdza. Trzy naturalne miejsca: po doprecyzowaniu wymagań, po planie i przed włączeniem zmian do głównego kodu. W projekcie od zera pierwszy punkt przesuwam później. Nie ma czego zepsuć, więc pozwalam asystentowi dojść do planu samemu, a ja czytam plan i końcową listę zmian. Czytanie planu to najtańszy sposób, żeby złapać zły kierunek: dziesięć linijek zamiast tysiąca. W istniejącym projekcie odwrotnie, zatwierdzam wcześniej, bo koszt złego kierunku rośnie z każdym modulem, który już istnieje.

I limity. Pętla „napisz, przetestuj, popraw” ma ograniczenie: trzy próby. Po trzeciej asystent nie kombinuje dalej, tylko zatrzymuje się i opisuje, co mu nie wychodzi: które testy, jaki błąd, co próbował.

To jest dokładnie ten moment z historii na początku. Czwarta, piąta próba, kiedy oczywiste poprawki się skończyły, to moment, w którym asystent zaczyna szukać nieoczywistych. Na przykład zmiany testu. Limit zamienia asystenta, który idzie na skróty, w asystenta, który prosi o pomoc.

+ OPCJONALNIE

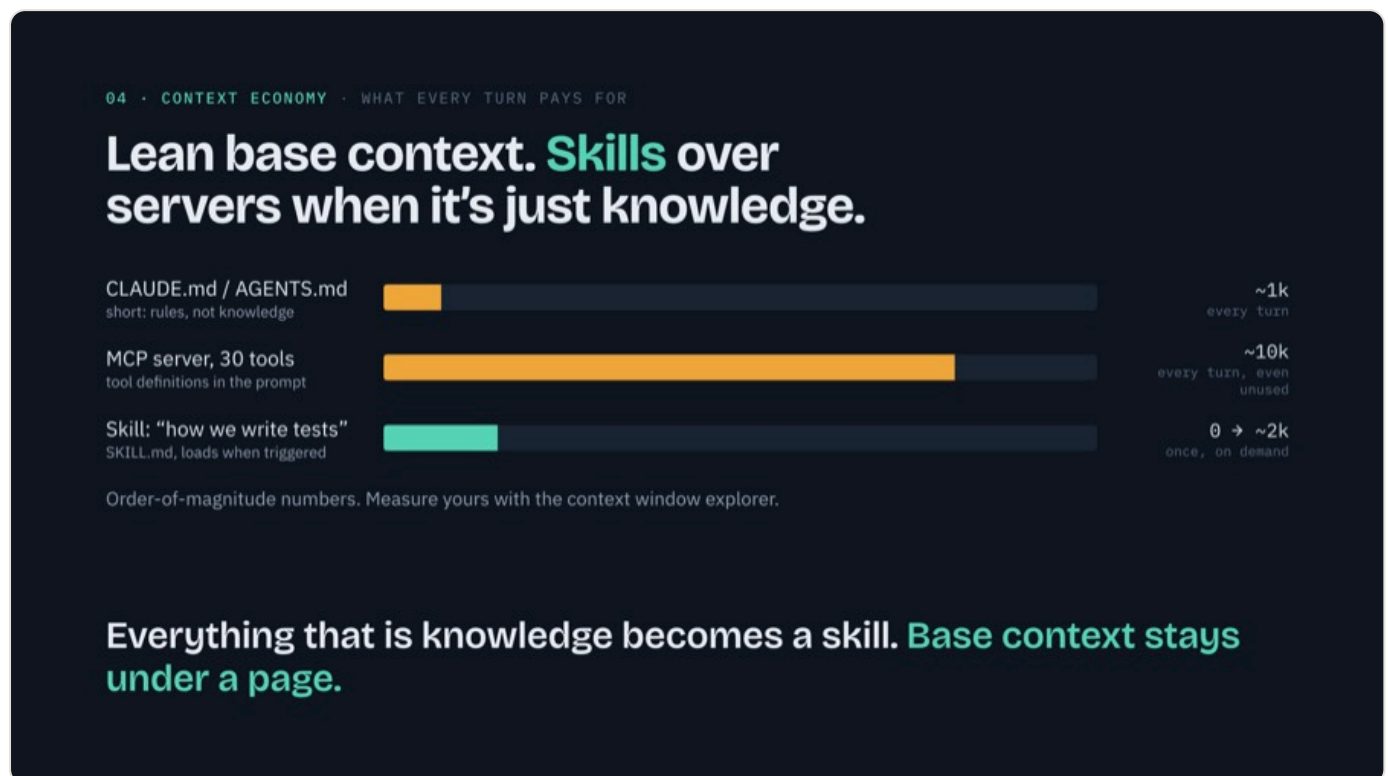
Trzy to nie jest magiczna liczba. Chodzi o to, żeby limit istniał i żeby był mniejszy niż liczba prób, po której asystent zaczyna być kreatywny. U mnie to zwykle trzy, czasem dwa.

+ OPCJONALNIE

Punkt zatwierdzenia to też naturalne miejsce, żeby zacząć od czystej pamięci. O tym za chwilę.

09. Lean base context

slajd 1:15 · do końca slajdu 10:30



Druga część: oszczędne gospodarowanie pamięcią asystenta. Bo odseparowanie etapów nic nie da, jeśli każdy etap startuje z pięćdziesięcioma stronami balastu, zanim przeczyta pierwsze zadanie.

Zasada pierwsza: krótkie instrukcje bazowe. Plik, który asystent czyta na starcie każdej rozmowy, ma być krótki. Reguły, nie wiedza. „Nie zmieniaj testów” to reguła, jedna linijka. „Jak u nas piszemy testy” to wiedza, dwie strony. Wiedza idzie do instrukcji na żądanie i wczytuje się dopiero wtedy, kiedy asystent zaczyna zadanie, którego dotyczy.

Zasada druga: jeśli chodzi tylko o wiedzę, instrukcja na żądanie zamiast wtyczki z narzędziami. Wtyczka kosztuje przy każdym kroku, bo asystent za każdym razem dostaje opis wszystkich jej narzędzi. Trzydzieści narzędzi to kilka, kilkanaście stron tekstu, płatne przy każdym kroku, nawet jeśli asystent nie użyje żadnego. Instrukcja na żądanie nie kosztuje nic, dopóki nie zostanie użyta.

Liczby na slajdzie to rzędy wielkości, nie pomiar. Zmierzcie swoje. Narzędzia mają do tego podgląd, ile pamięci co zajmuje, link na końcu.

+ OPCJONALNIE

Prosty test: jeśli wtyczka, którą macie podpiętą, tylko odpowiada na pytania „jak coś zrobić”, to powinna być instrukcją. Wtyczka ma sens, kiedy asystent musi coś wykonać: odpytać bazę, założyć zgłoszenie, przeszukać repozytorium.

+ OPCJONALNIE

Jeden z najczęstszych błędów, jakie widzę: plik z instrukcjami, który urósł do 400 linii, bo po każdym incydencie ktoś dopisał regułę. Po trzech miesiącach asystent czyta go jak regulamin basenu. Czyli wcale.

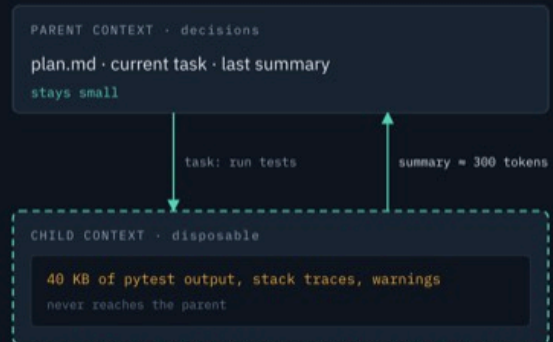
10. Subagents as context firewalls

slajd 1:00 · do końca slajdu 11:30

04 · CONTEXT ECONOMY · TOOLS ON A DIET

Efficient MCP. Subagents as context firewalls.

- **Few tools, well described.** The description is the interface.
- **Cap result sizes.** A tool that returns 40 KB is a context leak.
- **Tool search** for large sets: load definitions on demand.
- **Targeted code search** over dumping files into the prompt.



Exploration and test runs happen in a child context. Only the summary comes back.

Kiedy wtyczka z narzędziami jest potrzebna, bo asystent musi coś zrobić, a nie tylko wiedzieć, trzymam ją na diecie. Cztery zasady po lewej.

Mało narzędzi, dobrze opisanych, bo opis narzędzia to jedyne, co asystent o nim wie. Limit rozmiaru odpowiedzi: narzędzie, które zwraca czterdzieści stron, zapycha pamięć jednym wywołaniem. Wyszukiwanie narzędzi zamiast wczytywania wszystkich opisów naraz, jeśli narzędzi jest dużo. I precyzyjne wyszukiwanie w kodzie zamiast wrzucania całych plików.

Drugi mechanizm, po prawej: pomocnicy jako zaporę. Przeglądanie repozytorium, uruchamianie testów: to robi pomocnik z własną, osobną pamięcią, która po zadaniu jest wyrzucana. Do głównego asystenta wraca tylko streszczenie: „3 testy nie przeszły, oto błąd, oto plik”, a nie czterdzieści stron surowego wyjścia. Główna pamięć zostaje czysta na decyzje.

+ OPCJONALNIE

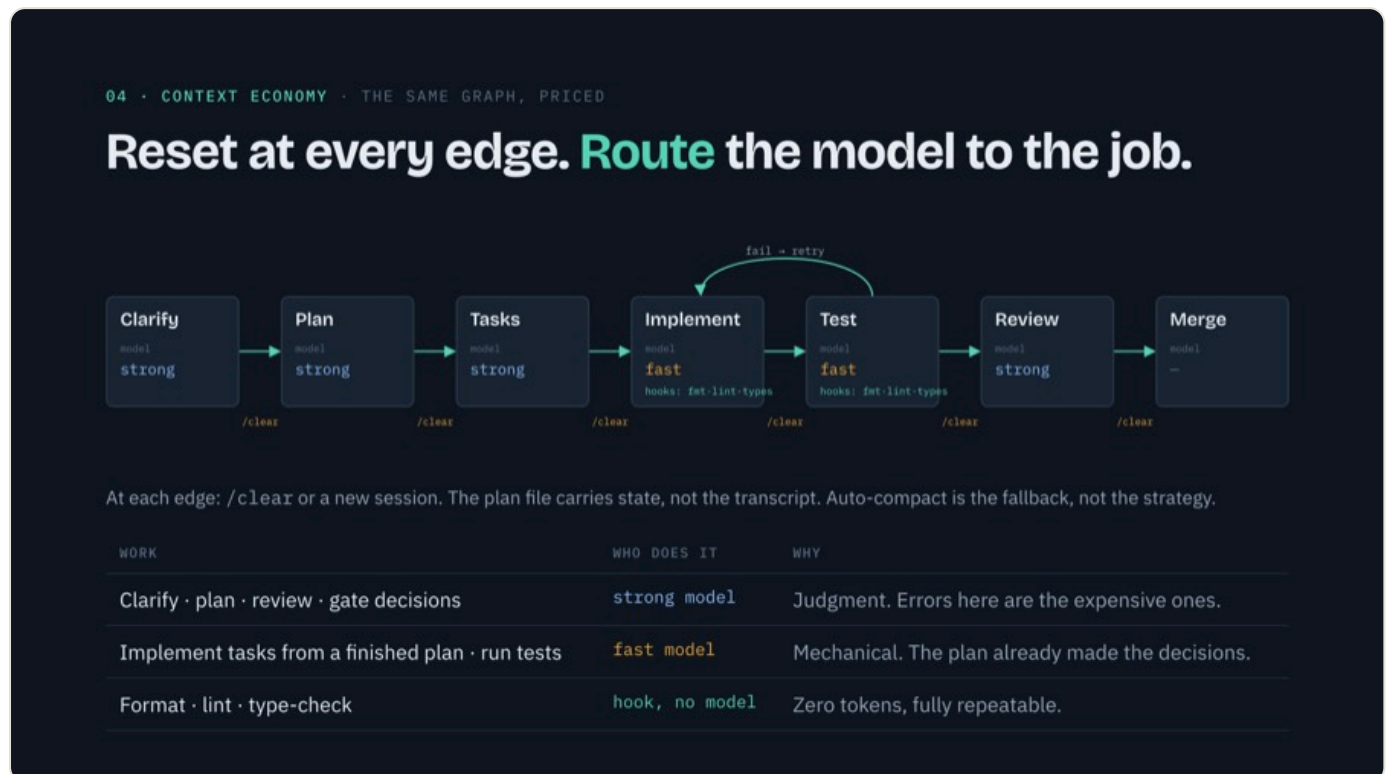
To ten sam pomysł co odseparowanie etapów, tylko w mniejszej skali. Etap też może mieć swoich pomocników, którzy nie zaśmiecają mu pamięci.

+ OPCJONALNIE

Praktyczna wskazówka: jeśli widzicie, że główny asystent czyta piętnasty plik z rzędu „żeby zrozumieć strukturę”, to jest moment na pomocnika. Główny asystent potrzebuje mapy, nie całego terenu.

11. Reset between phases, route models

slajd 1:30 · do końca slajdu 13:00



Czysta pamięć między etapami. Na każdej strzałce: nowa rozmowa. Stan przenosi plik z planem, nie historia rozmowy. To wymusza dyscyplinę: jeśli decyzji nie ma w pliku, to jej nie było.

Automatyczne streszczanie długiej rozmowy, które oferują narzędzia, to zabezpieczenie awaryjne, nie metoda pracy. Streszczenie gubi właśnie te szczegóły, na których zależy najbardziej. Na przykład to, że warunek numer trzy mówi o błędzie, a nie o przesunięciu.

I dobór modelu do zadania. Mocny model tam, gdzie są decyzje: doprecyzowanie wymagań, planowanie, sprawdzanie, zatwierdzanie. Tam błąd jest drogi, bo przenosi się na wszystko, co dalej. Tańszy i szybszy model tam, gdzie praca jest mechaniczna: pisanie kodu według gotowego planu, uruchamianie testów. Plan już podjął decyzje, wykonawca ma je zrealizować.

A tam, gdzie w ogóle nie trzeba sztucznej inteligencji, formatowanie kodu, sprawdzanie stylu, sprawdzanie typów: zwykły skrypt uruchamiany automatycznie. Zero kosztu, sto procent powtarzalności. Nie proście asystenta, żeby „pamiętał o formatowaniu”. Niech formatowanie uruchamia się samo po każdym zapisie.

To jest ta sama mapa etapów co wcześniej, tylko każdy etap ma etykietę z ceną.

+ OPCJONALNIE

W praktyce różnica w koszcie między „wszystko na najmocniejszym modelu” a takim doborem to kilka razy. A jakość na etapach mechanicznych nie spada, bo tam nie ma czego decydować.

12. Same graph, two configurations

slajd 1:15 · do końca slajdu 14:15

05 · TWO CONFIGURATIONS · ON SPEC-KIT

Same graph, two configurations

Greenfield

timezone CLI · nothing to break yet

GATES	Late: after plan, before merge
MODEL	Fast model on implementation tasks
TESTS	Written first, from the spec, before any code
WRITE SCOPE	Wide: all of src/
KNOWLEDGE	Constitution + one testing skill

Brownfield

Hermes dashboard · years of conventions

GATES	Early: after clarify, after plan, before merge
MODEL	Strong model longer; fast only on isolated tasks
TESTS	Existing suite is the fence; new tests cite criteria
WRITE SCOPE	Narrow: one module per run
KNOWLEDGE	Conventions extracted into skills first; code-search MCP

Same nodes. Different parameters and gate positions.

Ta sama mapa etapów, dwa ustawienia. Nie zmieniam procesu. Zmieniam parametry etapów i miejsca, gdzie zatwierdza człowiek.

Projekt od zera, nasz program do stref czasowych: zatwierdzanie późno, po planie i przed włączeniem zmian. Tańszy model na zadaniach, bo plan jest dobry, a zadania małe. Testy pisane najpierw, na podstawie specyfikacji, zanim powstanie linijka programu. Szeroki dostęp do zapisu, bo nie ma czego zepsuć. Wiedza: krótkie zasady projektu plus jedna instrukcja o tym, jak piszemy testy.

Istniejący projekt, powiedzmy panel z kilkuletnią historią: najpierw spisuję zwyczaje z istniejącego kodu do instrukcji na żądanie, żeby asystent pisał tak, jak pisze zespół, a nie tak, jak pisze internet. Wtyczka do przeszukiwania kodu, żeby rozumieć projekt bez czytania wszystkiego. Zatwierdzanie wcześniej: po wymaganiach, po planie, przed włączeniem. Wąski dostęp do zapisu, jeden moduł na raz. Istniejące testy są ogrodzeniem, nowe testy odwołują się do warunków.

To jest cały sens myślenia etapami: proces jest ustawialny, a nie budowany od nowa dla każdego projektu.

+ OPCJONALNIE

Jeśli mielibyście zabrać z tego slajdu jedną rzecz do istniejącego projektu: spisanie zwyczajów zespołu do instrukcji przed pierwszą linijką kodu. To najtańsza inwestycja z najwyższym zwrotem.

13. Close

slajd 0:30 · do końca slajdu 14:45

06 · CLOSE

**Spec-kit gives
you nodes. You
engineer the
edges, the budgets,
and the fences.**

The rogue agent was not a model
problem. It was a topology
problem: one node, every
permission, no stop condition.

Jedno zdanie na koniec. Narzędzia dają wam etapy. Strzałki, limity i ogrodzenia ustawiacie sami.

Asystent z początku, ten, który poprawił testy zamiast programu, nie był problemem modelu. Był problemem ustawienia: jeden etap, wszystkie uprawnienia, brak limitu prób. Zmieńcie ustawienie, a ten sam model stanie się przewidywalny.

+ OPCJONALNIE

I to jest, moim zdaniem, właściwa definicja inżynierii grafów: nie dziesięciu asystentów naraz, tylko świadome decyzje o tym, co każdy z nich widzi, co może dotknąć i kiedy ma się zatrzymać.

14. References

slajd 0:30 · do końca slajdu 15:15

REFERENCES · HUBS ON THE SLIDE, DEEP LINKS IN THE NOTES

Where to read next

CLAUDE CODE

code.claude.com/docs/en/features-overview · when to use CLAUDE.md, skills, subagents, hooks, MCP
.../hooks-guide · .../hooks · the fence mechanism
.../skills · .../sub-agents · .../mcp
.../context-window · .../costs · .../model-config
.../workflows · .../best-practices

SPEC-KIT

github.com/github/spec-kit · community walkthroughs

GITHUB COPILOT CLI

docs.github.com/.../copilot-cli/customize-copilot · instructions, hooks, skills, MCP, custom agents
docs.github.com/.../customize-github-copilot-app

OPENAI CODEX

developers.openai.com/codex/concepts/customization · AGENTS.md, memories, skills, MCP, subagents
.../codex/mcp · .../codex/concepts/subagents

Linki do dokumentacji: narzędzia Anthropic, GitHuba i OpenAI oraz spec-kit. Te same mechanizmy, instrukcje, instrukcje na żądanie, blokady, wtyczki, pomocnicy, istnieją w każdym z nich. Różni się tylko zapis. Slajdy będą dostępne, nie przepisujcie.

Dziękuję. Chętnie odpowiem na pytania.

Dla siebie · *Możliwe pytania. „Skąd te liczby?” Rzędy wielkości; każde narzędzie ma podgląd zużycia pamięci. „A równoległość?” Najpierw odseparowanie; równoległość dostajecie za darmo, kiedy etapy są niezależne. „Czy blokada nie przeszkadza też ludziom?” Blokada działa tylko w sesji asystenta; człowiek edytuje w swoim edytorze normalnie. „Czy to nie za dużo ceremonii na małe zadanie?” Tak. Na jednorazowy skrypt wystarczy polecenie. Etapy opłacają się, kiedy proces ma się powtarzać albo koszt błędu jest wysoki. „Co, jeśli tester i wykonawca to ten sam model?” Nie szkodzi. Chodzi o osobną pamięć i osobne uprawnienia, nie o różne modele.*

Łącznie 15:15 dla tekstu podstawowego. Opcjonalne akapity dodają około 4 minut, jeśli przeczytasz wszystkie.